# Lab 2 - HDFS Hands-on Exercises

# Lab Report

**Course:** Big Data Processing Technology
**Lab:** HDFS Hands-on Exercises
**Institution:** Beijing Institute of Technology
**Major:** School of Computer Science, Data Science and Big Data Technology (Chinese) - Class 07152301
**Name:** 吴雨桐 (Wu Yutong)
**Student ID:** 1120230572
**Date:** 2026-03-19

## 1. Experimental Environment

- **OS:** CentOS 7 (VMware Workstation 17 Pro)
- **JDK Version:** OpenJDK 1.8.0_461
- **Hadoop Version:** Hadoop 3.1.3
- **Cluster Architecture:** Includes one Master node (master) and one Worker node (node1).
- **Other Components:** ZooKeeper 3.5.8 and HBase 2.2.7 are configured.

## 2. HDFS Commands

Description: Step-by-step execution of HDFS commands with codes, results, and screenshots.

## 2.1 Preparation

**Task:** Start the cluster and prepare test data

**Command:**
Bash

```
# 启动 HDFS
start-dfs.sh

# 查看 Java 进程，确认 NameNode、DataNode 等服务已启动
jps
```

**Result & Screenshot:**
**Start HDFS in the master terminal and check the status using jps**

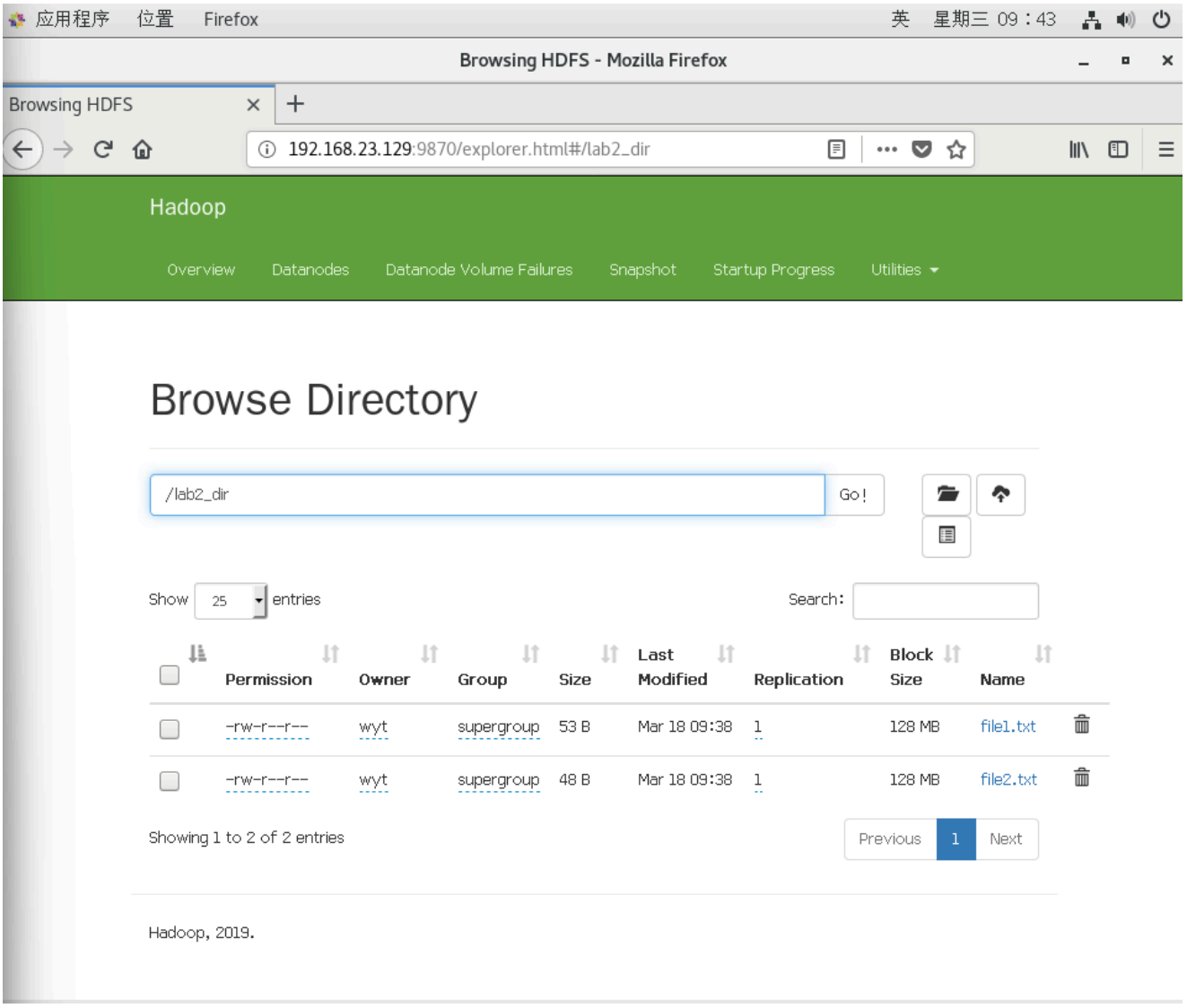
**Command:**

Bash

```
# 在本地创建一个临时文件夹进入
mkdir -p ~/lab2_test
cd ~/lab2_test

# 创建两个测试文本文件
echo "Hello Hadoop! This is the first test file for Lab 2." > file1.txt
echo "HDFS stands for Hadoop Distributed File System." > file2.txt

# 在 HDFS 中创建一个专门用于本次实验的目录
hdfs dfs -mkdir -p /lab2_dir

# 将本地刚建好的两个文件上传到 HDFS 中
hdfs dfs -put file1.txt /lab2_dir/
hdfs dfs -put file2.txt /lab2_dir/
```

**Result & Screenshot:**

**View the test files created in HDFS via the browser at port 192.168.23.129:9870**



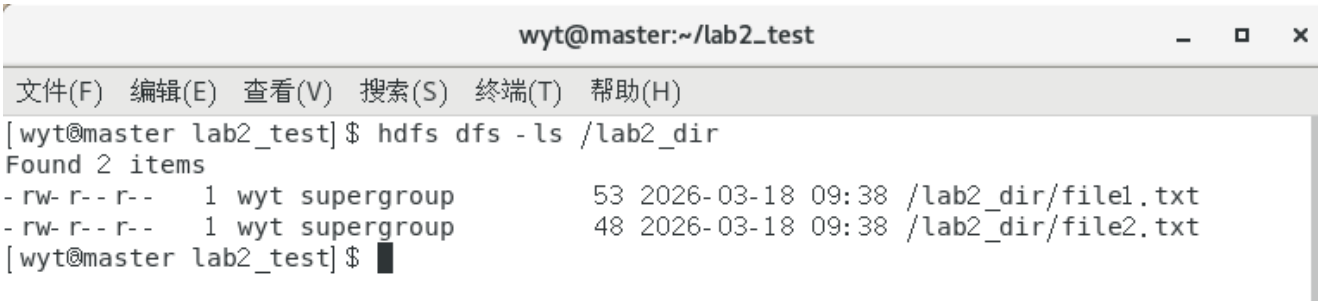

## 2.2 Directory Information

**Task:** Output read and write permissions, file size, and creation time of all files in a given HDFS directory.

**Command:**
Bash

```
# 查看刚才创建的 /lab2_dir 目录下的文件信息
hdfs dfs -ls /lab2_dir
```

**Result & Screenshot:**



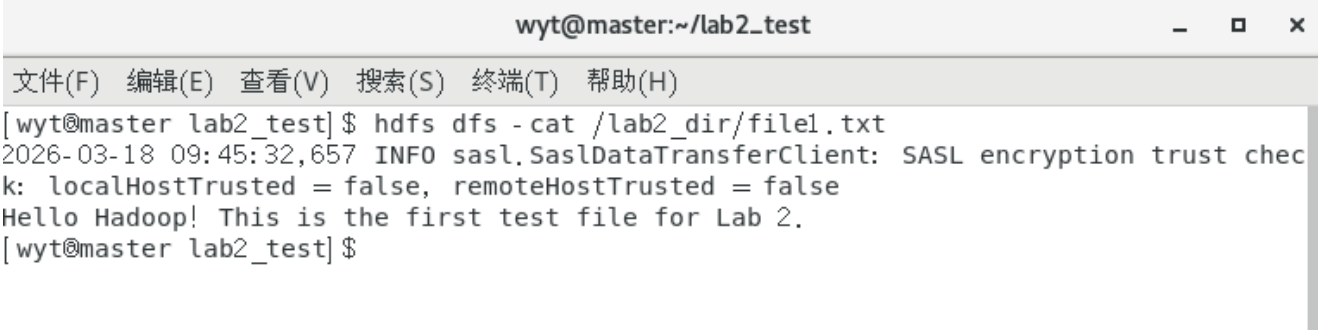
## 2.3 Display File Content

**Task:** Display the content of a file in HDFS.

**Command:**
Bash

```
hdfs dfs -cat /lab2_dir/file1.txt
```

**Result & Screenshot:**




## 2.4 Copy and Append

**Task:** Copy a text file from the local file system to HDFS; if it exists, append the content to the end of the existing file.

**Command:**
Bash

```
# 1. 先在本地新建一个包含追加内容的文本文件
echo "This is the appended text from local system." > append_file.txt

# 2. 将本地的 append_file.txt 追加到 HDFS 中已存在的 file1.txt 末尾
hdfs dfs -appendToFile append_file.txt /lab2_dir/file1.txt

# 3. 再次查看 file1.txt 的内容，验证追加是否成功
hdfs dfs -cat /lab2_dir/file1.txt
```

**Result & Screenshot:**

```
wyt@master:~/lab2_test
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
wyt@master lab2_test]$ echo "This is the appended text from local system." > append
_file.txt
wyt@master lab2_test]$ hdfs dfs -appendToFile append_file.txt /lab2_dir/file1.txt
2026-03-18 09:46:49,132 INFO sasl.SaslDataTransferClient: SASL encryption trust chec
k: localHostTrusted = false, remoteHostTrusted = false
wyt@master lab2_test]$ hdfs dfs -cat /lab2_dir/file1.txt
2026-03-18 09:46:54,722 INFO sasl.SaslDataTransferClient: SASL encryption trust chec
k: localHostTrusted = false, remoteHostTrusted = false
Hello Hadoop! This is the first test file for Lab 2.
This is the appended text from local system.
wyt@master lab2_test]$
```

## 2.5 Download File

**Task:** Copy a file from HDFS to the local file system.

**Command:**
Bash

```
# 将 HDFS 中的 file2.txt 下载到当前的本地目录，并命名为 downloaded_file2.txt 以示区分
hdfs dfs -get /lab2_dir/file2.txt ./downloaded_file2.txt

# 查看本地当前目录的文件，确认是否下载成功
ls -l downloaded_file2.txt

# 查看下载到本地的文件内容，验证数据是否完整
cat downloaded_file2.txt
```

**Result & Screenshot:**

```
wyt@master:~/lab2_test
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
[wyt@master lab2_test]$ hdfs dfs -get /lab2_dir/file2.txt ./downloaded_file2.txt
2026-03-18 09:47:46,265 INFO sasl.SaslDataTransferClient: SASL encryption trust chec
k: localHostTrusted = false, remoteHostTrusted = false
[wyt@master lab2_test]$ ls -l downloaded_file2.txt
-rw-r--r-- 1 wyt wyt 48 3月  18 09:47 downloaded_file2.txt
[wyt@master lab2_test]$ cat downloaded_file2.txt
HDFS stands for Hadoop Distributed File System.
[wyt@master lab2_test]$
```

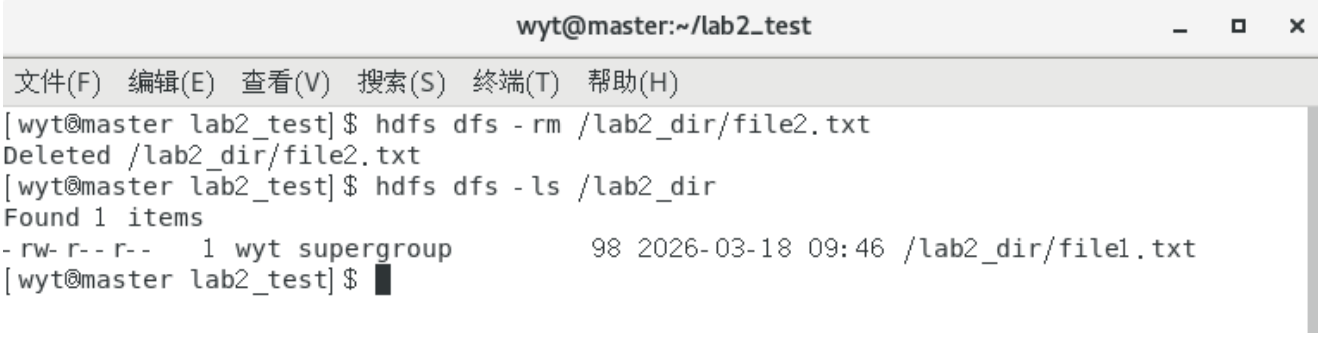
## 2.6 Delete File

**Task:** Delete a file in HDFS.

**Command:**
Bash

```
# 1. 删除 HDFS 中的 file2.txt
hdfs dfs -rm /lab2_dir/file2.txt

# 2. 再次查看 /lab2_dir 目录，确认 file2.txt 已经被删除（此时应该只剩下
file1.txt）
hdfs dfs -ls /lab2_dir
```

**Result & Screenshot:**




# 3. Hadoop Java API Programming

Description: Implementation of Java programs using the Hadoop Java API.

## 3.1 Interactive Upload

**Task:** Copy a text file from local file system to HDFS; if the file already exists, let the user specify whether to append or overwrite.

**Command:**
Bash

```
# 1.创建 java 文件
nano InteractiveUpload.java

# 2. 打包成 Jar 文件
jar -cvf InteractiveUpload.jar InteractiveUpload.class

# 3. 第一次执行（测试直接上传）
hadoop jar InteractiveUpload.jar InteractiveUpload my_java_test.txt
/lab2_dir/my_java_test.txt

# 4. 修改本地文件内容（准备用于追加的新内容）
echo "Second line: This is appended via Java" > my_java_test.txt

# 5. 第二次执行（测试交互逻辑）
hadoop jar InteractiveUpload.jar InteractiveUpload my_java_test.txt
/lab2_dir/my_java_test.txt
```

**Code:**

Java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IOUtils;

import java.io.FileInputStream;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.URI;
import java.util.Scanner;

public class InteractiveUpload {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: hadoop InteractiveUpload <local_path>
<hdfs_path>");
            System.exit(1);
        }

        String localStr = args[0];
        String hdfsStr = args[1];

        // 获取 Hadoop 配置并强制开启追加支持
        Configuration conf = new Configuration();
        conf.setBoolean("dfs.support.append", true);
        FileSystem fs = FileSystem.get(URI.create(hdfsStr), conf);

        Path localPath = new Path(localStr);
        Path hdfsPath = new Path(hdfsStr);

        // 核心逻辑：判断 HDFS 中是否已存在该文件
        if (fs.exists(hdfsPath)) {
            System.out.println("The file " + hdfsStr + " already exists in
HDFS.");
            System.out.print("Do you want to [A]ppend or [O]verwrite? (Enter
A or O): ");

            Scanner scanner = new Scanner(System.in);
            String choice = scanner.nextLine().trim().toUpperCase();

            if ("A".equals(choice)) {
                System.out.println("Appending to existing file...");
                InputStream in = new FileInputStream(localStr);
                OutputStream out = fs.append(hdfsPath);
                IOUtils.copyBytes(in, out, 4096, true);
                System.out.println("Append successful!");
```

```
            } else if ("O".equals(choice)) {
                System.out.println("Overwriting existing file...");
                fs.copyFromLocalFile(false, true, localPath, hdfsPath);
                System.out.println("Overwrite successful!");
            } else {
                System.out.println("Invalid choice. Operation cancelled.");
            }
            scanner.close();
        } else {
            System.out.println("File does not exist in HDFS. Uploading...");
            fs.copyFromLocalFile(localPath, hdfsPath);
            System.out.println("Upload successful!");
        }
        fs.close();
    }
}
```

**Result & Screenshot:**

```
wyt@master lab2_test]$ jar -cvf InteractiveUpload.jar InteractiveUpload.class
已添加清单
正在添加: InteractiveUpload.class(输入 =2647) (输出 =1450)(压缩了 45%)
wyt@master lab2_test]$ hadoop jar InteractiveUpload.jar InteractiveUpload my_java_t
est.txt /lab2_dir/my_java_test.txt
File does not exist in HDFS. Uploading...
2026-03-18 09:54:55,687 INFO sasl.SaslDataTransferClient: SASL encryption trust chec
k: localHostTrusted = false, remoteHostTrusted = false
Upload successful!
wyt@master lab2_test]$ echo "Second line: This is appended via Java" > my_java_test
.txt
wyt@master lab2_test]$ hadoop jar InteractiveUpload.jar InteractiveUpload my_java_t
est.txt /lab2_dir/my_java_test.txt
The file /lab2_dir/my_java_test.txt already exists in HDFS.
Do you want to [A]ppend or [O]verwrite? (Enter A or O): A
Appending to existing file...
2026-03-18 09:55:17,634 INFO sasl.SaslDataTransferClient: SASL encryption trust chec
k: localHostTrusted = false, remoteHostTrusted = false
Append successful!
wyt@master lab2_test]$
```

## 3.2 Download with Auto-Rename

**Task:** Copy a file from HDFS to local file system; if there is a file with the same name locally, rename it automatically.

**Command:**
Bash

```bash
# 1.创建 java 文件
nano AutoRenameDownload.java

# 2. 编译并打包
# 编译源代码
javac -cp $(hadoop classpath) AutoRenameDownload.java
```

```
# 打包成 Jar 文件
jar -cvf AutoRenameDownload.jar AutoRenameDownload.class
```

**Code:**

Java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;

import java.io.File;
import java.net.URI;

public class AutoRenameDownload {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: hadoop jar <jar_name>
AutoRenameDownload <hdfs_path> <local_path>");
            System.exit(1);
        }

        String hdfsStr = args[0];
        String localStr = args[1];

        Configuration conf = new Configuration();
        FileSystem fs = FileSystem.get(URI.create(hdfsStr), conf);
        Path hdfsPath = new Path(hdfsStr);

        // 检查 HDFS 文件是否存在
        if (!fs.exists(hdfsPath)) {
            System.out.println("Error: The file " + hdfsStr + " does not
exist in HDFS.");
            System.exit(1);
        }

        File localFile = new File(localStr);
        String finalLocalPath = localStr;

        // 核心逻辑：如果本地文件已存在，则自动重命名
        if (localFile.exists()) {
            System.out.println("The file " + localStr + " already exists
locally.");
            int dotIndex = localStr.lastIndexOf('.');
            String namePart;
            String extPart = "";

            if (dotIndex != -1) {
                namePart = localStr.substring(0, dotIndex);
```

```
                extPart = localStr.substring(dotIndex);
            } else {
                namePart = localStr;
            }

            int count = 1;
            File newLocalFile;
            // 循环寻找未被占用的文件名，例如 file_1.txt, file_2.txt
            do {
                finalLocalPath = namePart + "_" + count + extPart;
                newLocalFile = new File(finalLocalPath);
                count++;
            } while (newLocalFile.exists());

            System.out.println("Auto-renaming the downloaded file to: " +
finalLocalPath);
        }

        Path localDownloadPath = new Path(finalLocalPath);

        // 下载文件到本地 (delSrc=false, useRawLocalFileSystem=true 避免 CRC 文
件干扰)
        fs.copyToLocalFile(false, hdfsPath, localDownloadPath, true);
        System.out.println("Download successful! Saved as: " +
finalLocalPath);

        fs.close();
    }
}
```

**Result & Screenshot:**

```
[wyt@master lab2_test]$ nano AutoRenameDownload.java
[wyt@master lab2_test]$ javac -cp $(hadoop classpath) AutoRenameDownload.java
[wyt@master lab2_test]$ jar -cvf AutoRenameDownload.jar AutoRenameDownload.class
已添加清单
正在添加: AutoRenameDownload.class(输入 = 2226) (输出 = 1224)(压缩了 45%)
[wyt@master lab2_test]$ hadoop jar AutoRenameDownload.jar AutoRenameDownload /lab2_dir/my
_java_test.txt ./my_java_test.txt
The file ./my_java_test.txt already exists locally.
Auto-renaming the downloaded file to: ./my_java_test_1.txt
2026-03-18 10:00:30,320 INFO sasl.SaslDataTransferClient: SASL encryption trust check: lo
calHostTrusted = false, remoteHostTrusted = false
Download successful! Saved as: ./my_java_test_1.txt
[wyt@master lab2_test]$ ls -l my_java_test*
-rw-r--r-- 1 wyt wyt 78 3月  18 10:00 my_java_test_1.txt
-rw-rw-r-- 1 wyt wyt 39 3月  18 09:55 my_java_test.txt
[wyt@master lab2_test]$
```

## 4. Problems and Solutions

- **Problem 1:** When trying to access the HDFS Web UI via `http://localhost:9870` from the physical machine's browser, it showed "Unable to connect".

    - **Solution 1:** Realized that `localhost` in the physical machine's browser points to the physical machine itself, not the CentOS virtual machine. Used the `ip addr` command in the VM terminal to find its actual IP address, and successfully accessed the Web UI using `http://<VM_IP>:9870`. Also verified that the CentOS `firewalld` service was disabled to prevent port blocking.
- **Problem 2:** When attempting to run the compiled Java program directly using `hadoop InteractiveUpload`, the terminal threw an error: `ERROR: InteractiveUpload is not COMMAND nor fully qualified CLASSNAME`.
    - **Solution 2:** In Hadoop 3.x, running bare `.class` files can sometimes cause classpath resolution issues. The standard and most robust solution was to package the compiled `.class` file into a JAR archive using `jar -cvf InteractiveUpload.jar InteractiveUpload.class`, and then execute it via the `hadoop jar` command.
- **Problem 3:** Encountered a `bash: !": event not found` error when using the `echo` command to create a local text file containing an exclamation mark ( `!` ).
    - **Solution 3:** The exclamation mark is a special character in Bash used for history expansion. Solved this by removing the exclamation mark from the string, or it could also be solved by using single quotes instead of double quotes for the string.

## 5. References


1. HDFS commands: http://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-common/FileSystemShell.html
2. Hadoop Java API: http://hadoop.apache.org/docs/stable/api/index.html